

UNIVERSIDADE FEDERAL DE RONDÔNIA - UNIR

Programa Institucional de Iniciação Científica - PIBIC

ESPECIFICAÇÃO DO APLICATIVO DE GERENCIAMENTO DAS VMs DO PIBIC

Base técnica para o “PIBIC LAB / PIBIC Access”

Marinaldo da Costa Barroso Filho

PIBIC/UNIR - Projeto PVN2111-2026 - Banco de Dados, interoperabilidade e o uso de Inteligência Artificial em dados institucionais

Data de referência: 09 de setembro de 2026

Porto Velho - RO
2026

Resumo executivo

O aplicativo proposto deve esconder a complexidade de ZeroTier, SSH, port forwarding e Proxmox sem remover os controles de segurança. O MVP recomendado é um desktop em Python com interface HTML/CSS/JavaScript, terminal xterm.js e backend AsyncSSH. O núcleo deve ser independente da camada visual para permitir uso do Vela Framework no MVP e eventual migração para PySide6/QtWebEngine se houver necessidade de maior maturidade de empacotamento. O aplicativo não deve transportar senhas comuns do laboratório, não deve embutir token administrativo do ZeroTier Central e não deve operar o Proxmox como root. Metadados ficam em SQLite; segredos ficam no keyring nativo do sistema operacional.

1. Contexto e objetivo

A infraestrutura atual validada possui uma VM Alpine gateway acessível por ZeroTier em 10.57.144.217. A interface Web do Proxmox 10.99.0.59:8006 é alcançada por SSH local forwarding para 127.0.0.1:18006. O aplicativo deve automatizar essa sequência e, no futuro, permitir que cada integrante veja apenas a VM e os recursos que lhe foram atribuídos.

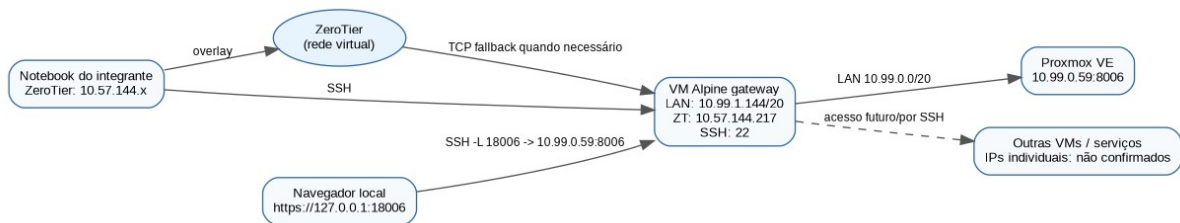


Figura 1 - Infraestrutura existente que o aplicativo deve abstrair.

2. Perfis de usuário

Perfil	Poderes esperados	Restrições
Administrador do laboratório	Autorizar dispositivos, cadastrar servidores/VMs, definir permissões, configurar integração Proxmox, aprovar manifests e jobs administrativos.	Não deve usar root como credencial padrão do aplicativo; ações críticas exigem confirmação/auditoria.
Pesquisador/Aluno PIBIC	Acessar sua VM, terminal, informações do sistema e serviços permitidos; abrir Proxmox somente conforme autorização.	Não pode listar/operar VMs de outros usuários nem ganhar wheel/root por meio do app.
Professor/Orientador	Perfil configurável: auditoria, visão dos recursos do projeto e ações administrativas aprovadas.	Privilégios devem ser explícitos, não inferidos pelo nome/cargo.

3. Requisitos funcionais

ID	Requisito
RF-01	Cadastrar conexões de laboratório e VMs sem salvar senha em texto puro.
RF-02	Detectar instalação/estado do ZeroTier e exibir Node ID para autorização quando necessário.
RF-03	Estabelecer sessão SSH com validação de host key/fingerprint.
RF-04	Abrir terminal SSH interativo com PTY, resize, Ctrl+C, programas full-screen e cores.

ID	Requisito
RF-05	Criar/encerrar port forwarding local para Proxmox e outros serviços autorizados.
RF-06	Consultar API do Proxmox com API Token de privilégio mínimo.
RF-07	Listar somente VMs permitidas ao perfil e mostrar status/CPU/RAM/disco/IP/uptime quando disponível.
RF-08	Iniciar/desligar/reiniciar VM somente quando a role permitir e após confirmação.
RF-09	Detectar SO, kernel, CPU, RAM, disco, Git, Python, Node.js, PostgreSQL, pgVector, Nginx, Apache, SSH, Docker/Compose e outros detectores cadastrados.
RF-10	Detectar containers Docker, imagem, estado, portas, volumes e projetos Compose quando permitido.
RF-11	Exibir catálogo versionado de serviços com detecção, compatibilidade e ações assistidas.
RF-12	Antes de qualquer instalação, mostrar plano, comandos, necessidade de privilégio e pedir confirmação explícita.
RF-13	Registrar jobs, resultado, versão do manifest e usuário que executou a ação.
RF-14	Atualizar aplicativo e catálogo por mecanismos separados.
RF-15	Permitir diagnóstico/exportação de logs sem incluir segredos.

4. Requisitos não funcionais

- Segurança por padrão: menor privilégio, host key verification, segredos fora do SQLite e nenhuma credencial administrativa hardcoded.
- Disponibilidade degradável: se ZeroTier estiver TUNNELED, o app deve informar latência/relay em vez de tratar como erro fatal.
- Portabilidade: Windows e Linux como prioridade; macOS em seguida.
- Auditabilidade: toda ação administrativa deve gerar registro local e, quando possível, ID da task do Proxmox.
- Testabilidade: serviços SSH, Proxmox, detecção e catálogo devem ser abstraídos por interfaces e testáveis com mocks.
- Recuperação: jobs de instalação precisam registrar estado e suportar verificação/rollback quando o manifest definir rollback.
- Usabilidade: um aluno deve conseguir “Conectar -> Terminal” sem conhecer IP/porta depois do cadastro inicial.

5. Arquitetura recomendada

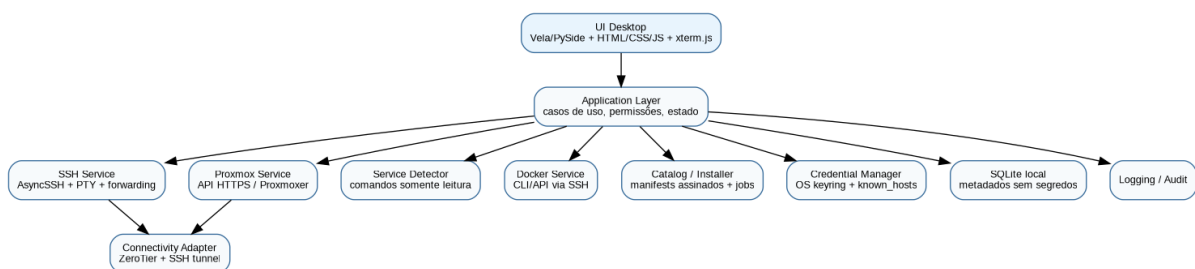


Figura 2 - Arquitetura lógica proposta.

Camada/componente	Responsabilidade
UI	Navegação, dashboard, terminal, formulários, confirmação e feedback. Não contém lógica de credenciais ou comandos administrativos.
Application Layer	Casos de uso: ConnectVM, OpenTerminal, OpenProxmox, DetectServices, InstallService. Aplica autorização antes dos serviços.
SSH Service	Conexões AsyncSSH, PTY, SFTP, comandos seguros e TCP forwarding.
Proxmox Service	Cliente API, tasks assíncronas, status de VM e operações permitidas.
Docker Service	Inspeção e ações de Docker via SSH/CLI no MVP; API/SDK em fase futura.
Service Detector	Executa detectores somente leitura, normaliza saída e produz DetectedService.
Package Installer	Transforma manifest em InstallationJob; mostra plano; executa etapas; verifica pós-condição.
Credential Manager	Integra keyring do SO, host keys, chaves SSH e tokens de API.
Manifest Repository	Catálogo local versionado, assinatura/checksum e atualização independente.
Update Manager	Atualização do aplicativo e atualização do catálogo como fluxos distintos.
Logging/Audit	Eventos estruturados, sem senha/token; correlação por job/session/task.
SQLite	Servidores, VMs conhecidas, preferências, catálogo cacheado, histórico e estado; nunca senha em claro.

6. Avaliação de tecnologias e stack recomendada

Tecnologia	Vantagens	Desvantagens	Decisão
Vela Framework + HTML/CSS/JS	Alinha com projetos existentes do desenvolvedor; Python no backend; facilita UI Web e xterm.js.	Framework menor/pessoal: maturidade, documentação e empacotamento precisam ser validados.	Bom para MVP se o core não depender do Vela.
PySide6 + QtWebEngine	Ecossistema Qt maduro; integra WebView para xterm.js; bom acesso a recursos desktop.	Binários maiores; packaging e WebEngine aumentam distribuição.	Fallback recomendado se o Vela limitar terminal/IPC.
Electron + backend Python	xterm.js e WebSocket naturais.	Muito pesado e duplica runtime; mais superfície de atualização.	Não recomendado para o MVP.
Tauri + sidecar Python	Binário UI leve e moderno.	Mais linguagens/toolchain; comunicação sidecar e permissões aumentam complexidade.	Considerar somente depois do MVP.
AsyncSSH	Nativo asyncio, SSHv2, SFTP, PTY e forwarding [R16].	Requer arquitetura assíncrona correta.	Recomendado.
Paramiko	Biblioteca SSH Python madura [R17].	API mais síncrona/baixo nível para terminal em tempo real.	Fallback; não primeira escolha.
Proxmoxer	Wrapper Python para APIs Proxmox e backends HTTPS/SSH [R21].	É camada de conveniência; ainda exige compreender API e segurança TLS.	Recomendado sobre API oficial, com token restrito.
SQLite	Banco local sem servidor [R19].	Não serve para segredos nem concorrência multiusuário pesada.	Recomendado para metadados.

Tecnologia	Vantagens	Desvantagens	Decisão
keyring	Usa Credential Locker/Keychain/Secret Service [R20].	Backends variam por SO/desktop.	Recomendado para segredos locais.

Stack proposta para o MVP: Python 3.12+; Vela (se estável no ambiente) com frontend HTML/CSS/JS; xterm.js; AsyncSSH; Proxmoxer + requests/httpx; SQLite; keyring; Pydantic para modelos/validação; logging estruturado; PyInstaller/empacotador compatível com o Vela. O núcleo deve expor interfaces, permitindo trocar Vela por PySide6 sem reescrever SSH/Proxmox/catalog.

7. Tela inicial / login e conectividade ZeroTier

A tela inicial não deve pedir “senha do laboratório” genérica. O usuário seleciona seu perfil e uma conexão cadastrada. O app verifica o ZeroTier por subprocesso controlado ou integração local, sem enviar comandos arbitrários fornecidos pela UI.

PIBIC LAB - Conectar

Ambiente: [Laboratório PIBIC / UNIR v]

Usuário SSH: [carlos.evangelista]

Método: [Chave SSH v] [Gerenciar chave]

ZeroTier: • OK / TUNNELED IP local: 10.57.144.xx

Gateway: • Disponível 10.57.144.217:22

[Conectar] [Diagnóstico]

- Se ZeroTier não estiver instalado: mostrar link/assistente de instalação oficial; não bundlear driver de rede ocultamente.
- Se não estiver na rede: permitir “Entrar na rede” com o Network ID conhecido e mostrar Node ID para aprovação.
- Nunca embutir API token do ZeroTier Central no cliente, pois isso permitiria autorizar dispositivos sem controle do administrador.
- Mostrar estados: OFFLINE, ONLINE, TUNNELED, ACCESS_DENIED e OK PRIVATE com explicações.

8. Terminal SSH integrado



Figura 3 - Fluxo do terminal interativo.

O terminal precisa de PTY remoto. Executar `conn.run("comando")` não atende vim, top, passwd, shells interativos ou programas que dependem de tamanho de terminal. AsyncSSH suporta shell channels, terminal type, tamanho e forwarding [R16]. O frontend xterm.js oferece eventos onData, resize e renderização de sequências de terminal [R18].

```
# Pseudocódigo conceitual - backend Python
async with asyncssh.connect(host, username=user, client_keys=keys,
known_hosts=known_hosts) as conn:
    proc = await conn.create_process(
        term_type='xterm-256color',
        term_size=(cols, rows),
        encoding=None,
    )
    # bytes do xterm.js -> proc.stdin
    # proc.stdout/stderr -> xterm.js
    # evento resize -> proc.change_terminal_size(cols, rows)
```

O IPC pode ser WebSocket ligado somente em 127.0.0.1 ou, se o Vela oferecer bridge nativa segura, uma ponte local sem socket de rede. Cada mensagem deve ter session_id, tipo (input/resize/close) e payload limitado. Nunca exponha o WebSocket em 0.0.0.0 por padrão.

9. Integração Proxmox

A aba Proxmox deve usar a API oficial e não automação de navegador. O pvedaemon expõe a API localmente e o pveproxy a disponibiliza externamente [R3]. O Proxmox possui modelo de usuários, roles, ACLs e API tokens; a documentação mostra que tokens podem ter privilégios separados e limitados a um subconjunto do usuário [R1].

Na topologia atual, o aplicativo pode primeiro abrir um local forwarding interno para a API, por exemplo 127.0.0.1:<PORTA_DINAMICA> -> 10.99.0.59:8006, e então usar HTTPS/Proxmoxer nesse endpoint. Em produção, preserve validação TLS: em vez de verify_ssl=False, importe/confie na CA/certificado correto ou faça pinning controlado.

Dado/ação	Fonte/API proposta	Permissão mínima sugerida
Listar nós/VMs	Proxmox API / cluster resources ou nodes/qemu	PVEAuditor ou role custom somente leitura.
Status CPU/RAM/uptime	status/current da VM	Auditoria na VM/pool atribuído.
Start/Stop/Reboot	endpoints status/start, stop, reboot	Role custom apenas nas VMs/pool do aluno ou ação mediada pelo admin.
Console	API de console/proxy conforme tipo	Somente se necessário; terminal SSH é preferível para VM Linux.
Configuração global	Não expor ao aluno	Somente administrador do laboratório.

Segurança: Nunca entregar ao aplicativo um token root@pam com acesso total. Crie usuário técnico dedicado e API Token com privilege separation/ACL no menor caminho possível (VM, pool ou read-only).

10. Detecção automática do ambiente

Os detectores devem usar comandos somente leitura e argumentos fixos. A UI nunca deve concatenar texto do usuário dentro de shell=True. Preferir execução por argv ou comandos predefinidos no backend.

Item	Detector sugerido	Observação
SO/distribuição	cat /etc/os-release	parse chave=valor.
Kernel	uname -r	string.
Hostname	hostname	string.
CPU	nproc; /proc/cpuinfo	normalizar número/modelo.
RAM	free -b ou /proc/meminfo	fallback por distribuição.

Item	Detector sugerido	Observação
Disco	df -B1 -P /	bytes total/usado/livre.
Git	command -v git && git --version	presente + versão.
Python	command -v python3 && python3 --version	presente + versão.
Node.js	command -v node && node --version	presente + versão.
Docker	command -v docker && docker --version	não executar container.
Compose	docker compose version	requer Docker CLI.
PostgreSQL	command -v psql && psql --version	não conectar a banco sem credencial.
pgVector	consulta de extensão somente se houver conexão DB autorizada	Não inferir apenas por arquivo/pacote.
Nginx/Apache	command -v + status do serviço	systemd/OpenRC adapter.
SSH server	ss -lnt + serviço sshd	não expor configuração secreta.

11. Docker

No MVP, o Docker Service pode executar a CLI remotamente por SSH e parsear saídas estruturadas (--format). Isso evita expor o socket Docker pela rede. O acesso ao daemon Docker é altamente privilegiado; em muitos ambientes, pertencer ao grupo docker equivale de fato a poder obter root no host. Portanto, o aplicativo não deve adicionar usuários ao grupo docker automaticamente.

Informação	Comando estruturado possível
Containers	docker ps -a --format <template JSON/TSV controlado>
Imagens	docker images --format <template>
Portas	docker inspect <ID> com formato/JSON
Volumes	docker inspect <ID> / docker volume ls
CPU/RAM	docker stats --no-stream --format <template>
Compose	docker compose ls --format json; docker compose ps --format json quando suportado

O backend deve validar IDs/names por whitelist de caracteres ou, melhor, passar argumentos separados para a biblioteca SSH, nunca interpolar texto em "sh -c". Para ações start/stop/restart, exibir alvo exato e exigir confirmação.

12. Catálogo/repositório de serviços

Cada serviço deve ser descrito por um manifest declarativo mantido no repositório oficial do projeto. O manifest não deve ser um script arbitrário baixado e executado. Ele descreve detectores, compatibilidade e etapas pré-aprovadas que o aplicativo traduz para operações permitidas.

```
# Exemplo conceitual de manifest YAML
id: docker
name: Docker Engine
category: containers
manifest_version: 1
supported:
  - os: debian
    versions: ["13"]
  - os: alpine
    versions: ["3.23"]
detect:
  argv: ["docker", "--version"]
install_plan:
  debian:
    steps:
      - action: apt_install
        packages: ["docker-ce", "docker-ce-cli", "containerd.io"]
verify:
  argv: ["docker", "--version"]
documentation: "https://docs.docker.com/"
requires_admin: true
```

- Campos recomendados: id, nome, descrição, categoria, versões suportadas, detectores, dependências, ações de instalação/upgrade/remoção, verificação, rollback, documentação, ícone e nível de privilégio.
- Não aceitar “command: curl URL | sh” como manifest genérico.
- Use ações tipadas: apt_install, apk_add, write_config_template, systemd_enable, openrc_enable, docker_pull etc.
- Manifests precisam de schema versionado e validação Pydantic/JSON Schema.
- O catálogo oficial deve ser assinado: release/tag assinada, checksum do pacote e chave pública embutida no aplicativo. Atualização só é ativada após verificação.

13. Atualização do aplicativo x atualização do catálogo

Fluxo	Aplicativo	Catálogo
Conteúdo	Binários Python/UI, dependências e migrações.	Manifests e metadados de serviços.
Frequência	Menor; requer instalação/rollback.	Pode ser maior; sem trocar binário.
Assinatura	Obrigatória; pacote de release assinado/checksum.	Obrigatória; índice + manifests versionados.
Rollback	Versão anterior do app + backup de DB/migração.	Manter N versões cacheadas e ponteiro current.
Permissão	Usuário/admin local.	Pode atualizar sem privilégios de sistema, mas nunca executar instalação silenciosa.

14. Instalação assistida de serviços



Figura 4 - Pipeline seguro de instalação assistida.

1. Detectar SO, versão, arquitetura e gerenciador de serviços/pacotes.
2. Carregar manifest compatível e validar assinatura/schema.
3. Executar detectores e pré-condições sem alterar o sistema.
4. Mostrar ao usuário os pacotes/arquivos/serviços que serão modificados.

5. Pedir confirmação; para ações administrativas, pedir elevação remota apenas no momento necessário.
6. Executar as etapas com stream de progresso no terminal/job view.
7. Executar verificação pós-instalação definida no manifest.
8. Salvar resultado, versão instalada e logs sanitizados; se falhar, oferecer rollback apenas quando especificado e seguro.

Segurança: Nunca permitir execução silenciosa de comandos administrativos importantes. A confirmação precisa mostrar o alvo e o efeito, não apenas um botão genérico “OK”.

15. Dashboard

PIBIC LAB	Ambiente: UNIR	Usuário: marcos
Dashboard	VM: <VM_DO_USUARIO>	● Conectada
Terminal	Hostname: pesquisa-xx	S0: Debian 13
Proxmox	CPU 12% RAM 2.1/8 GB	Disco 28/80 GB
Serviços	IP: <IP_DA_VM>	Uptime: 3d 04h
Docker		
Banco de dados	Docker: Instalado ✓	Containers ativos: 3
Logs	PostgreSQL: Instalado	pgVector: Detectado ✓
Arquivos		
Configurações	[Abrir terminal] [Abrir Proxmox] [Atualizar]	

Campos que não forem detectáveis não devem ser inventados. Exibir “Não disponível” ou “Permissão insuficiente”.

16. Wireframes das principais telas

Terminal:

Terminal - VM <nome>	SSH ● Conectado	Latência 180 ms
law@vm:~\$		
[xterm.js ocupa a área; seleção, copy/paste, resize, Ctrl+C]		
Sessão: 00:23:10	[Nova aba] [SFTP] [Desconectar]	

Proxmox:

Proxmox - Recursos autorizados						
VMID	Nome	Estado	CPU	RAM	Uptime	Ações
123	pesquisa-marcos	RUN	8%	2G	3d 4h	SSH ...
[Atualizar] Ações não permitidas ficam ocultas/desabilitadas.						

Serviços:

Serviços disponíveis		
Docker	Não instalado	[Ver plano] [Instalar]
Git	2.xx ✓	[Atualizar]
PostgreSQL	17.x ✓	[Gerenciar]
pgVector	Não confirmado	[Detectar]
Nginx	Não instalado	[Ver plano] [Instalar]

17. Banco de dados local

SQLite é apropriado para o MVP porque é embutido e não requer servidor [R19]. O banco guarda estado local não secreto. Segredos devem ser referenciados por um secret_ref que aponta para o keyring do SO [R20].

Tabela/entidade	Campos principais (conceituais)
Server	id, name, kind, gateway_host, ssh_port, proxmox_internal_host, enabled
VirtualMachine	id, server_id, vmid, name, node, assigned_user_profile_id, last_status
SSHConnection	id, host, port, username, auth_method, key_ref, known_host_fingerprint, last_seen
UserProfile	id, display_name, role, local_preferences
DetectedService	id, vm_id, service_id, installed, version, status, detected_at
DockerContainer	id, vm_id, docker_id, name, image, state, ports_json, volumes_json
ServiceManifest	id, version, checksum, source_revision, schema_version, enabled
InstallationJob	id, vm_id, manifest_id, requested_by, state, started_at, finished_at, exit_code, audit_ref

18. Segurança e modelo de ameaça

Área	Controle recomendado
Senhas	Não guardar em SQLite/config. Usar keyring e preferir chave SSH com passphrase.
Chaves SSH	Privada nunca sai do computador. Permissões corretas. Suportar agent. Public key pode ser cadastrada no servidor.
known_hosts/fingerprint	Strict host key checking. Primeiro cadastro deve exigir confirmação por canal confiável; alteração posterior é evento de segurança.
Proxmox API token	Token por aplicação/usuário técnico, ACL mínima, secret no keyring. Nunca root token global.
Command injection	Sem shell=True com entrada do usuário; argv e ações tipadas. Validar IDs/paths.
sudo/doas	Elevação sob demanda. Não armazenar senha de root. Comandos administrativos exibidos antes da execução.
Manifests	Schema, assinatura/checksum, origem oficial, allowlist de action types. Sem scripts remotos arbitrários.
Logs	Redação de Authorization, password, token, private key, env secrets e URLs com credenciais.

Área	Controle recomendado
Segregação	Mapeamento usuário->VM/pool; backend revalida permissão, não confiar apenas em UI oculta.
Atualização	Pacotes assinados, canal estável, rollback e migrações versionadas.
Terminal	xterm.js eleva o risco do app; WebSocket/IPC somente local, CSP restritiva e nenhuma origem externa no WebView [R18].
Docker	Não expor /var/run/docker.sock e não conceder grupo docker automaticamente.

19. Fluxo de conexão técnico

9. Usuário abre o aplicativo e seleciona ambiente/VM.
10. Connectivity Adapter verifica ZeroTier, autorização e gateway.
11. SSH Service resolve host/porta e consulta known_hosts.
12. Se a fingerprint for nova, o app apresenta SHA256 e pede confirmação; se tiver mudado, bloqueia por padrão.
13. Credential Manager fornece chave/senha a partir do keyring sem persistir no banco.
14. AsyncSSH autentica e abre sessão.
15. Service Detector coleta os dados somente leitura.
16. Application Layer aplica permissões e atualiza dashboard.
17. Terminal é habilitado com PTY. Proxmox e ações administrativas só aparecem se o perfil tiver autorização.
18. Ao desconectar, sessões, forwards e PTYs são encerrados e segredos são removidos da memória quando possível.

20. Estrutura de diretórios sugerida

```

pibic_lab/
├── pyproject.toml
├── src/pibic_lab/
│   ├── app.py
│   ├── domain/
│   │   ├── models.py
│   │   └── permissions.py
│   ├── application/
│   │   ├── connect_vm.py
│   │   ├── detect_services.py
│   │   └── install_service.py
│   ├── infrastructure/
│   │   ├── ssh/asyncssh_service.py
│   │   ├── proxmox/client.py
│   │   ├── zerotier/adaptador.py
│   │   ├── docker/service.py
│   │   ├── credentials/keyring_store.py
│   │   ├── database/sqlite.py
│   │   └── catalog/repository.py
│   ├── ui/
│   │   ├── views/
│   │   ├── static/
│   │   └── terminal/
│   ├── security/
│   │   ├── hostkeys.py
│   │   ├── manifest_verify.py
│   │   └── redaction.py
│   └── audit/
├── catalog/
│   ├── schema/
│   └── manifests/
├── tests/
│   ├── unit/
│   ├── integration/
│   └── fixtures/

```

Se Vela for usado, a pasta `ui/views/templates/static` pode seguir as convenções do framework, mas `domain/application/infrastructure` não deve importar Vela. Isso evita acoplamento e facilita testes.

21. Roadmap recomendado

Fase	Escopo	Critério
MVP 0 - fundação	Modelos, config, logging, SQLite, keyring, testes, adapter ZeroTier.	Sem UI avançada.
MVP 1 - conexão SSH	Cadastro de conexão, fingerprint, autenticação, terminal xterm.js + AsyncSSH PTY.	Primeira entrega utilizável.
MVP 2 - dashboard	Deteção de SO/CPU/RAM/disco e serviços básicos.	Somente leitura.
MVP 3 - Proxmox read-only	Túnel automático + API token mínimo; listar VM/status.	Sem start/stop inicialmente.
MVP 4 - ações Proxmox	Start/stop/reboot com confirmação e auditoria.	ACL por VM/pool.
MVP 5 - Docker	Detectar Docker/containers e logs; depois start/stop.	Sem instalação automática ainda.
MVP 6 - catálogo	Manifests assinados, detectores e planos de instalação.	Somente "ver plano" + dry-run primeiro.

Fase	Escopo	Critério
MVP 7 - instalação assistida	Jobs privilegiados com confirmação, verificação e rollback definido.	Segurança reforçada.
MVP 8 - update/plugins	Atualização separada app/catalog, extensões verificadas.	Após modelo de confiança maduro.

22. Funcionalidades futuras (fora do MVP)

- Integração assistida com PostgreSQL e pgVector, sem armazenar senhas de banco em SQLite.
- Jupyter com port forwarding temporário e URL/token tratados como segredo.
- Catálogo de serviços de IA/RAG e health checks.
- Logs agregados e observabilidade da VM/containers.
- SFTP/arquivos com sandbox por home do usuário.
- Backups, snapshots e restore via Proxmox com política/role específica.
- Relay ZeroTier próprio ou outra conectividade institucional autorizada.
- Integração com inventário e assignment de VMs por usuário.

23. Diagramas Mermaid de referência

Os diagramas abaixo podem ser mantidos no repositório junto da documentação e renderizados por ferramentas Mermaid.

```

```mermaid
graph TD
 subgraph TB
 UI[UI Desktop / xterm.js] --> APP[Application Layer]
 APP --> SSH[SSH Service / AsyncSSH]
 APP --> PVE[Proxmox Service / API]
 APP --> DET[Service Detector]
 APP --> DOCKER[Docker Service]
 APP --> CAT[Manifest Catalog / Installer]
 APP --> CREDS[Credential Manager / OS keyring]
 APP --> DB[(SQLite)]
 SSH --> ZT[ZeroTier + SSH tunnel]
 PVE --> ZT
 end
```

```

```

```mermaid
sequenceDiagram
 participant U as Usuário
 participant X as xterm.js
 participant B as Backend Python
 participant S as AsyncSSH
 participant V as VM
 U->>X: tecla / resize
 X->>B: IPC local
 B->>S: stdin / change_terminal_size
 S->>V: canal SSH + PTY
 V-->>S: stream do terminal
 S-->>B: bytes
 B-->>X: write(bytes)
```

```

```

```mermaid
graph LR
 A[Detectar S0] --> B[Validar manifest/assinatura]
 B --> C[Mostrar plano]
 C --> D{Usuário confirma?}
 D -- Não --> X[Cancelar]
 D -- Sim --> E[Executar job]
 E --> F[Verificar pós-condição]
 F --> G[Atualizar status + auditoria]
```

```

24. O que estudar para implementar

| Tema | Prioridade | Documentação |
|----------------------------------|------------|---|
| SSH/PTY/forwarding | Alta | OpenSSH [R4][R5], RFC 4251/4254 [R22][R23], AsyncSSH [R16]. |
| xterm.js e terminal sequences | Alta | xterm.js [R18]. |
| Proxmox API/RBAC/tokens | Alta | Proxmox Guide [R1], API Viewer e Proxmoxer [R21]. |
| Python asyncio | Alta | Fundamental para múltiplas sessões e streaming sem bloquear UI. |
| Credential storage | Alta | keyring [R20], SSH host keys. |
| SQLite/modelagem | Média | sqlite3 [R19]. |
| Docker | Média | Docker docs [R14][R15]. |
| Assinatura de releases/manifests | Média/Alta | Ed25519/minisign/Sigstore conforme decisão de projeto. |
| Empacotamento desktop | Média | Validar Vela/PyInstaller; fallback PySide6. |

25. Resultado esperado

Ao concluir o roadmap, o integrante deverá instalar o aplicativo, entrar na rede ZeroTier, aguardar autorização, selecionar a VM e clicar em “Conectar”. O aplicativo valida host key, autentica por chave/senha protegida pelo keyring, coleta dados do sistema e habilita terminal e ações conforme o perfil. O administrador mantém controle sobre ZeroTier, Proxmox API Token, ACLs, catálogo assinado e permissões. O aplicativo simplifica a operação, mas não transforma conveniência em privilégio administrativo.

Referências técnicas

[R1] PROXMOX SERVER SOLUTIONS GMBH. Proxmox VE Administration Guide. Disponível em: <https://pve.proxmox.com/pve-docs/pve-admin-guide.pdf>. Acesso em 9 set. 2026.

[R2] PROXMOX SERVER SOLUTIONS GMBH. Proxmox Cluster File System (pmxcfs). Disponível em: <https://pve.proxmox.com/pve-docs/chapter-pmxcfs.html>. Acesso em 9 set. 2026.

[R3] PROXMOX SERVER SOLUTIONS GMBH. pvedaemon(8) - PVE API Daemon. Disponível em: <https://pve.proxmox.com/pve-docs/pvedaemon.8.html>. Acesso em 9 set. 2026.

[R4] OPENBSD PROJECT. ssh(1) - OpenSSH remote login client. Disponível em: <https://man.openbsd.org/ssh.1>. Acesso em 9 set. 2026.

[R5] OPENBSD PROJECT. sshd_config(5) - OpenSSH daemon configuration. Disponível em: https://man.openbsd.org/sshd_config.5. Acesso em 9 set. 2026.

- [R6] **ALPINE LINUX.** Alpine Package Keeper (APK). Disponível em: https://wiki.alpinelinux.org/wiki/Alpine_Package_Keeper. Acesso em 9 set. 2026.
- [R7] **ALPINE LINUX.** OpenRC. Disponível em: <https://wiki.alpinelinux.org/wiki/OpenRC>. Acesso em 9 set. 2026.
- [R8] **ALPINE LINUX.** Setting up a SSH server. Disponível em: https://wiki.alpinelinux.org/wiki/Setting_up_a_SSH_server. Acesso em 9 set. 2026.
- [R9] **DEBIAN PROJECT.** Debian Documentation. Disponível em: <https://www.debian.org/doc/>. Acesso em 9 set. 2026.
- [R10] **DEBIAN PROJECT.** Debian Reference. Disponível em: <https://www.debian.org/doc/manuals/debian-reference/>. Acesso em 9 set. 2026.
- [R11] **ZEROTIER, INC..** ZeroTier CLI. Disponível em: <https://docs.zerotier.com/cli/>. Acesso em 9 set. 2026.
- [R12] **ZEROTIER, INC..** TCP Relay. Disponível em: <https://docs.zerotier.com/relay/>. Acesso em 9 set. 2026.
- [R13] **TAILSCALE INC..** Using Tailscale with your firewall. Disponível em: <https://tailscale.com/docs/integrations/firewalls>. Acesso em 9 set. 2026.
- [R14] **DOCKER, INC..** Docker Documentation. Disponível em: <https://docs.docker.com/>. Acesso em 9 set. 2026.
- [R15] **DOCKER, INC..** Docker Compose CLI reference. Disponível em: <https://docs.docker.com/reference/cli/docker/compose/>. Acesso em 9 set. 2026.
- [R16] **ASYNCSH PROJECT.** AsyncSSH documentation. Disponível em: <https://asyncssh.readthedocs.io/en/stable/>. Acesso em 9 set. 2026.
- [R17] **PARAMIKO PROJECT.** Paramiko documentation. Disponível em: <https://docs.paramiko.org/en/stable/>. Acesso em 9 set. 2026.
- [R18] **XTERM.JS PROJECT.** xterm.js documentation. Disponível em: <https://xtermjs.org/docs/>. Acesso em 9 set. 2026.
- [R19] **PYTHON SOFTWARE FOUNDATION.** sqlite3 - DB-API 2.0 interface for SQLite databases. Disponível em: <https://docs.python.org/3/library/sqlite3.html>. Acesso em 9 set. 2026.
- [R20] **PYTHON KEYRING PROJECT.** keyring documentation. Disponível em: <https://keyring.readthedocs.io/en/latest/>. Acesso em 9 set. 2026.
- [R21] **PROXMOXER PROJECT.** Proxmoxer documentation. Disponível em: <https://proxmoxer.github.io/docs/latest/>. Acesso em 9 set. 2026.
- [R22] **IETF.** RFC 4251 - The Secure Shell (SSH) Protocol Architecture. Disponível em: <https://www.rfc-editor.org/rfc/rfc4251>. Acesso em 9 set. 2026.
- [R23] **IETF.** RFC 4254 - The Secure Shell (SSH) Connection Protocol. Disponível em: <https://www.rfc-editor.org/rfc/rfc4254>. Acesso em 9 set. 2026.